

IDEA TO PROTOTYPE – Three-Model Teardown

Sonnet 5, DeepSeek Flash, and GPT OSS 120B on Groq, tested on the same pipeline

Meghna Bhagwat | Product Manager | [LinkedIn](#)

How I Tested

My [Idea to Prototype tool](#) turns a one-line product idea into a clickable prototype in two AI steps. The first, the spec step, writes a product spec. The second, the code step, turns that spec into working code for the prototype.

I first tested these steps within one provider, comparing two Claude models, Sonnet 5 and Haiku 4.5, to see whether each step needs the same model. This teardown extends that test across providers. Sonnet 5 is a premium closed model. DeepSeek Flash is a low-cost open-weight model. GPT OSS 120B runs on Groq, on hardware built for speed.

Each model ran the same 5 product ideas twice with the same prompts. For the code step, all three models worked from the same spec for each idea, the one Sonnet 5 wrote in the Claude comparison. That way, any difference between prototypes came from how each model wrote the code. I reused Sonnet 5's specs and prototypes from that comparison, then scored all 60 outputs blind in one sitting. Sonnet 5 and GPT OSS ran with their default reasoning. DeepSeek ran with thinking off, for reasons covered below. All DeepSeek runs used its off-peak pricing.

Quality

	Sonnet 5	DeepSeek Flash	GPT OSS 120B
Mean prototype score (1 to 5)	2.9	3.2	2.1
Prototypes that loaded	9 of 10	10 of 10	7 of 10
Specs approved without major edits	6 of 10	9 of 10	6 of 9
Spec screens specific to the idea	9 of 10	10 of 10	9 of 9

DeepSeek scored highest and never failed to load. With only 10 prototypes each, small gaps can happen by luck. DeepSeek's lead over GPT OSS is big enough to trust. Its lead over Sonnet is too small, so I count it as a tie. I left one GPT OSS spec unscored.

INSIGHT 1

The cheapest model matched the premium one on quality. DeepSeek tied Sonnet on prototypes and led on specs, even though my prompts were written and tuned on Claude.

Speed and Cost

	Sonnet 5	DeepSeek Flash	GPT OSS 120B
Median time, spec (seconds)	10.6	3.4	1.8
Median time, code (seconds)	38.7	10.7	5.6
Time to first token (seconds)	Not recorded	0.9	0.2
Cost per idea to prototype (USD)	\$0.065	\$0.0023	\$0.0022

Sonnet costs about 28 times more than DeepSeek at off-peak rates, and about 14 times more at peak. GPT OSS was the fastest by a wide margin. It hit Groq's free tier rate limit twice, and I excluded those waits from its times.

INSIGHT 2

At this quality level, paying more bought nothing measurable. The real choice is between DeepSeek's reliability and GPT OSS's speed, at almost identical cost.

What the Scores Don't Show

Reasoning Hurt More Than It Helped

In its two test runs with thinking on, DeepSeek spent 79% of its spec tokens and 91% of its code tokens on reasoning, ran out of room, and cut off mid-answer. With thinking off, it scored highest of the three. In this test, reasoning made writing specs and prototype code slower and more expensive, without making the results any better.

Automated Checks Missed Real Failures

My automated checks confirm that each prototype's code builds without errors. Two prototypes passed those checks and still broke when I used them. One came from Sonnet and wouldn't load because of a missing line of code. The other came from GPT OSS and crashed when I clicked a button.

Separately, three GPT OSS prototypes failed the checks. Generating each one again fixed all three, but I didn't score the new versions, so GPT OSS has no quality score for that setup yet.

The Spec Prompt Is the Shared Weak Spot

Across all three models, my notes returned to the same gaps. Specs didn't say where users create or add things, and navigation back to the main view was often missing.

INSIGHT 3

Picking a model is only one part of getting good results. How the model is set up, what it's given to work from, and how its output is checked all mattered just as much. Turning DeepSeek's thinking off took it from failing to scoring highest. Gaps in my spec prompt carried into every model's prototypes. And my checks let broken prototypes through, because they only confirmed the code could build. None of the checks opened a prototype to try it.

What I'd Build Next

Based on these results, I'd use DeepSeek Flash, with thinking off, as the default model for both steps. At my tool's traffic, routing across three providers would save only a few dollars a month, so I'd switch to DeepSeek alone and tell visitors where their input is processed. At production scale, the math changes. Sonnet would cost about \$65,000 per million prototypes, compared with about \$2,300 to \$4,600 for DeepSeek. At that volume, I'd route most requests to DeepSeek, send speed-critical ones to GPT OSS with a load check and one retry, and keep Sonnet as a fallback. Either way, I'd test each prototype by opening it in a browser and update the spec prompt to cover creation flows and back navigation.

PRODUCT RECOMMENDATION

Test model settings along with models and verify every prototype by loading it. For this pipeline, DeepSeek Flash with thinking off gives the best quality per dollar, with GPT OSS for speed and Sonnet as a fallback.

This was 10 runs per model per step, scored by one reviewer. DeepSeek's prices vary by time of day, and its models and pricing have changed often. GPT OSS's retried prototypes still need scoring.